Project Report

On

“DEVELOPMENT OF A FRAMEWORK FOR FAULT-TOLERANT DRONE FLIGHT CONTROL SYSTEM”

Submitted in partial fulfillment of the requirements for the

III Semester

Mini Project Work

Bachelor of Engineering in

Electronics and Communication Engineering

Of

Kalasalingam Academy of Research and Education

By

S. SRINIVASAN (9925005147)

SRIRAM. B (9925005150)

NAVANEETHAN. S (9925005153)

R. SHASHTHIKA. R. (9925005154)

Internal Guide
Name: _____
Assistant Professor
Dept. of ECE, KARE

External Guide
Name: _____
Designation: _____
Organisation: _____

Department of Electronics and Communication Engineering

Kalasalingam Academy of Research and Education

2026–2027

CERTIFICATE

This is to certify that S. SRINIVASAN (9925005147), SRIRAM. B (9925005150), NAVANEETHAN. S (9925005153) and R. SHASHTHIKA. R. (9925005154), bonafide students of Kalasalingam Academy of Research and Education, have completed the project entitled “Development of a Framework for Fault-Tolerant Drone Flight Control System” in partial fulfillment of the requirements for the III Semester Bachelor of Engineering in Electronics and Communication Engineering during the academic year 2026–2027.

It is certified that the project report has been prepared for academic evaluation. Any proposed parameters, illustrative calculations or simulated values are identified as such and shall be replaced by verified project observations before final submission.

Internal Guide	Head of the Department
Signature: _____	Signature: _____
Name: _____	Name: _____
Dept. of ECE, KARE	Dept. of ECE, KARE

External Viva

Name of the Examiners Signature with date

1. _____ _____

2. _____ _____

DECLARATION

We, the undersigned students, hereby declare that the project report entitled “Development of a Framework for Fault-Tolerant Drone Flight Control System” is prepared by our team as part of the III Semester Mini Project Work in Electronics and Communication Engineering at Kalasalingam Academy of Research and Education.

We further declare that the technical descriptions and proposed design parameters are presented for academic study. Any illustrative or simulated results are clearly identified and will not be represented as measured results. The work has not been submitted previously for any other degree or award.

Date: _____

Place: Krishnankoil

ABSTRACT

Unmanned aerial vehicles (UAVs), commonly called drones, depend on embedded flight-control systems to maintain stable attitude, estimate motion, and respond safely to pilot or mission commands. A fault in a sensor, processor, communication link or power subsystem can affect the quality of information used by the controller. This project presents a modular framework for monitoring system health and supporting fault-tolerant supervisory decisions in an educational multirotor platform.

The proposed framework is organised into sensing, state estimation, fault detection, fault isolation, control allocation, supervisory safety management and telemetry modules. It considers sensor redundancy, timestamp validation, range checks, stale-data detection, consistency monitoring and watchdog-based task supervision. When abnormal behaviour is detected, the monitor records the evidence, assigns a severity level and requests a predefined conservative response from the safety supervisor.

The report describes the problem, design objectives, system architecture, hardware and software requirements, representative fault models, detection logic, algorithms, test plan, evaluation metrics and future improvements. A software-in-the-loop and propeller-removed bench-testing approach is proposed before any supervised flight evaluation. Numerical values included as examples are illustrative planning values and must be verified against selected component datasheets and actual test logs.

Keywords: UAV, drone, fault tolerance, flight controller, IMU, sensor redundancy, fault detection, watchdog, failsafe.

TABLE OF CONTENTS

Chapter 1: Introduction

Chapter 2: Case Study and Problem Analysis

Chapter 3: Proposed Methodology and System Architecture

Chapter 4: Literature Survey

Chapter 5: Hardware and Software Implementation

Chapter 6: Flowchart and Algorithm

Chapter 7: Simulation, Testing and Result Analysis

Chapter 8: Conclusion and Future Scope

Chapter 9: References

Appendix A: Proposed Parameter Sheet

Appendix B: Test Record Template

Appendix C: Items to Complete Before Final Submission

Note: Update page numbers in the table of contents after final editing and insertion of the team's actual diagrams, photographs and test records.

CHAPTER 1: INTRODUCTION

1.1 Introduction

A drone is a compact cyber-physical system in which sensors, embedded processors, communication links, motors and propellers operate together to produce controlled flight. A multirotor typically uses an inertial measurement unit (IMU) to measure angular rate and acceleration. A flight controller combines these measurements with estimation and feedback-control algorithms to regulate roll, pitch and yaw. Position or altitude functions may additionally use a barometer, GNSS receiver, magnetometer, optical-flow sensor or range sensor.

The reliability of this system is important because flight-control decisions are made continuously. Sensor noise, loose connections, vibration, processor lockup, stale data, power dips, or actuator degradation can affect the control loop. Fault tolerance is the ability of a system to continue providing an acceptable level of service in the presence of specified faults, or to move to a safe condition when continued operation is not possible.

1.2 Problem Statement

Many low-cost drone prototypes are built around a single flight controller and a limited set of sensors. If a critical sensor becomes invalid, a data stream freezes, or the controller stops executing correctly, the system may not identify the problem early enough. The project therefore addresses the need for a structured framework that monitors critical components, identifies abnormal or missing data, manages degraded operating modes, and provides a predictable failsafe response.

1.3 Objectives

Develop a modular fault-tolerant architecture for a small multirotor flight-control system.

Identify representative sensor, processor, communication and power-related faults.

Implement data-validity checks using range, timeout, rate-of-change and sensor-consistency tests.

Provide supervisory logic for fault logging, fault classification and safe-mode selection.

Define a simulation and bench-test procedure using repeatable fault-injection scenarios.

Evaluate the framework using detection latency, false-alarm rate, data availability and safe-state transition success.

1.4 Scope of the Project

The scope is limited to a conceptual and prototype-level safety framework for a small educational multirotor. The work focuses on monitoring and supervisory logic rather than designing a high-performance autonomous navigation system. The report assumes that the base flight controller already provides conventional attitude stabilisation. The proposed framework adds health monitoring, redundancy management and failsafe supervision.

1.5 Limitations

The values in this report are design assumptions and must be checked against the selected flight controller and sensor datasheets.

Redundant sensors do not guarantee safety if they share the same power source, mounting fault, software defect or environmental disturbance.

A controlled descent or landing response is not guaranteed to be safe in every environment; the response must be selected according to the actual platform and operating rules.

Real flight testing requires qualified supervision, a suitable test area, regulatory compliance and a documented risk assessment.

1.6 Organisation of the Report

Chapter 2 analyses the fault-tolerance problem. Chapter 3 presents the architecture and methodology. Chapter 4 reviews relevant literature and standards. Chapter 5 describes proposed hardware and software. Chapter 6 gives the monitoring algorithm. Chapter 7 defines the simulation and test plan with example calculations. Chapter 8 concludes the work and lists future scope. Chapter 9 provides references.

1.7 Importance of the Study

A fault-tolerant design improves the dependability of an embedded system by making abnormal conditions visible before they develop into uncontrolled behaviour. In a drone, the health of sensing, computation, communication and power subsystems influences the reliability of the complete platform.

The educational value of this project is the integration of electronics, embedded programming, control-system concepts and software testing. It encourages a requirements-driven approach in which every monitored fault is linked to observable evidence, a defined response and a repeatable verification method.

CHAPTER 2: CASE STUDY AND PROBLEM ANALYSIS

2.1 Typical Drone Flight-Control Subsystems

Subsystem	Function	Representative fault
IMU (gyroscope + accelerometer)	Measures angular rate and specific force	Bias drift, saturation, frozen output
Barometer / altitude sensor	Provides altitude-related measurement	Pressure disturbance, offset, timeout
GNSS receiver	Provides global position and velocity	Signal loss, invalid fix, stale position
Flight controller	Runs estimation and control tasks	Processor lockup, task overrun, memory fault
ESC and motor	Convert control commands into rotor thrust	Command mismatch, motor/ESC failure
Battery and power distribution	Supplies regulated electrical power	Voltage sag, connector interruption
Telemetry link	Carries status and operator commands	Packet loss, delay, link timeout

2.2 Why Fault Tolerance Is Required

A conventional control loop assumes that sensor measurements are fresh and trustworthy. In practice, the system must distinguish between normal variations and abnormal behaviour. For example, a sensor may report a value within its numerical range but remain unchanged for an unusually long time. Similarly, two sensors may both produce plausible values while disagreeing beyond an allowed tolerance. Fault monitoring therefore uses more than one test.

2.3 Fault Classification

Fault class	Example	Proposed monitor
Hard fault	No sensor data or controller reset	Timeout and heartbeat monitor
Soft fault	Slow bias drift or increased noise	Trend and consistency check
Intermittent fault	Short bursts of invalid data	Consecutive-sample counter
Communication fault	Delayed or missing telemetry packet	Sequence number and age check
Power fault	Battery or regulated rail below limit	Voltage threshold and rate check
Actuator-response fault	Commanded response not reflected in feedback	Command/feedback plausibility monitor

2.4 Proposed Fault-Tolerant Strategy

The framework uses a layered strategy: (i) detect whether a data item is valid and recent, (ii) compare independent sources where available, (iii) classify the likely fault, (iv) select a permitted degraded mode, and (v) record the event for later analysis. The system should avoid switching modes because of a single noisy sample; persistence counters and hysteresis reduce unnecessary mode changes.

2.5 Design Requirements

ID	Requirement	Acceptance target (proposed)
R1	Monitor critical sensor freshness	Flag a missing IMU update within 100 ms
R2	Detect persistent sensor disagreement	Flag after 5 consecutive inconsistent samples
R3	Supervise controller execution	Watchdog heartbeat checked every 100 ms
R4	Log fault events	Record timestamp, source, fault class and action
R5	Enter a defined safe mode	Supervisor state changes within 200 ms after confirmed critical fault
R6	Avoid nuisance switching	Use threshold hysteresis and persistence counters

2.6 Failure Effects and Monitoring Priorities

The effect of a fault depends on its severity and the role of the affected subsystem. A missing telemetry packet may reduce operator awareness while the onboard controller remains functional; loss of a critical inertial measurement can directly affect confidence in attitude estimation. The framework therefore distinguishes advisory faults from faults that affect control integrity.

Monitoring priority should consider the consequence of the fault, the time available to recognise it and the availability of independent evidence. This prioritisation helps the project team focus limited processing and testing effort on important health indicators.

A fault log should preserve the first occurrence time, source identifier, measured value where appropriate, configured limit, persistence count and supervisor decision. Consistent records make it easier to reproduce a problem and compare results between test runs.

CHAPTER 3: PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Overall Architecture

The proposed system is divided into seven functional blocks: sensor acquisition, data validation, state estimation interface, fault detection, fault isolation, safety supervisor, and telemetry/event logging. The base flight controller remains responsible for the normal stabilisation loop. The safety supervisor observes health information and can request a predefined mode transition through the flight controller's supported failsafe interface.

Block	Input	Output / responsibility
Sensor acquisition	IMU, barometer, GNSS, battery status	Timestamped measurements
Data validation	Raw measurements and timestamps	Valid / invalid flags
State-estimation interface	Validated measurements	Estimated attitude/altitude status
Fault detector	Measurements, health flags, timing	Fault flags and confidence
Fault isolation	Fault flags and source identity	Suspected faulty component
Safety supervisor	Fault severity, flight state	Normal, degraded, recovery or disarm request
Telemetry and log	Health state and events	Operator display and stored record

3.2 Redundancy Concept

For critical measurements, the design may use two independent sensor channels or a primary sensor plus a dissimilar backup source. The framework does not blindly average all readings. Each channel is first checked for range, freshness and self-reported status. If two valid channels disagree, the supervisor marks the measurement as uncertain and applies the project's predefined response. Common-cause faults must be considered during physical placement, wiring and power design.

3.3 Data Validation

Each measurement is associated with a timestamp and validity flag. A sample is rejected if it is outside the sensor's configured physical range, older than the permitted age, non-finite, or unchanged for an implausibly long interval. Rate-of-change checks identify abrupt jumps that exceed a project-defined bound. These limits must be derived from the selected sensor datasheet and validated operating envelope.

3.4 Sensor Consistency Check

For two comparable channels x1 and x2, the disagreement is calculated as:

$$e = |x1 - x2|$$

A disagreement flag is raised when e exceeds a configured tolerance for a specified number of consecutive samples. For the proposed simulation, an illustrative threshold of 5 degrees per second for gyro-rate disagreement and five consecutive samples is used. This is only a starting point; actual limits depend on sensor range, noise, vibration and calibration.

3.5 Fault Severity and Supervisory States

State	Meaning	Supervisor action
NORMAL	All required channels valid	Permit nominal flight-control mode
DEGRADED	One non-critical source uncertain or unavailable	Use only validated sources; notify operator
RECOVERY	Critical fault confirmed but a controlled response remains available	Request preconfigured recovery mode and alert operator
DISARMED / SAFE	Control integrity cannot be assured or ground state is detected	Prevent motor arming / request safe shutdown as supported

3.6 Proposed Timing Parameters

Parameter	Proposed value	Reason / note
IMU data rate (simulation)	200 Hz	Representative educational control-system sampling assumption
Health-monitor task	50 Hz (20 ms)	Allows frequent checks without duplicating the main controller
IMU stale-data timeout	100 ms	Five 200 Hz sample periods
Heartbeat supervision	100 ms	Detects stalled monitoring task
Disagreement persistence	5 samples	Rejects a single transient discrepancy
Fault log update	On event + 1 Hz summary	Keeps event detail and periodic health status

3.7 Methodology Steps

Select a supported educational flight-controller platform and identify available health/status interfaces.

Create a list of faults and define expected detection evidence for each fault.

Implement timestamp, range, finite-value and timeout checks.

Implement consistency checks for redundant or cross-checked measurements.

Implement fault severity classification and supervisor state transitions.

Build a simulation model and inject sensor freeze, timeout, bias, packet delay and watchdog faults.

Record detection latency, false alarms and successful safe-state transitions.

Only after simulation and bench validation, plan supervised and legally compliant flight evaluation.

3.8 Data Flow and Module Interaction

At each monitoring cycle, the acquisition interface obtains health information and attaches a timestamp to every sample. The validation stage checks numerical validity, permitted range and sample age. Only validated information is passed to comparison and diagnostic stages; invalid samples remain recorded as evidence but are not treated as trustworthy measurements.

The fault manager combines individual flags with persistence counters and severity rules. The supervisor evaluates the current system state and selects only a response approved for the selected flight-controller platform. The event logger stores the decision and the telemetry interface makes health status available for review.

Separating these functions improves maintainability. A new sensor can be added to acquisition and validation without rewriting the event logger. The fault manager can also be verified using recorded data before connection to a physical platform.

CHAPTER 4: LITERATURE SURVEY

4.1 UAV Flight-Control Systems

Small UAV control architectures commonly combine inertial sensing, state estimation and feedback control. The PX4 and ArduPilot ecosystems document flight-stack architecture, sensor calibration, estimator status and failsafe configuration. These platforms provide useful reference material for an academic framework because they expose health and status information without requiring the project to replace the complete stabilisation controller.

4.2 Fault Detection and Diagnosis

Fault detection and diagnosis (FDD) methods are commonly grouped into model-based, signal-based and knowledge-based approaches. Model-based methods compare measured behaviour with a mathematical model. Signal-based methods use thresholds, trends, spectral characteristics or statistical tests. Knowledge-based methods use rules and expert-defined fault signatures. A small prototype can begin with transparent threshold, timeout and consistency checks before considering more complex estimation methods.

4.3 Redundancy and Sensor Fusion

Redundancy can improve availability when channels fail independently. However, redundancy must be designed carefully because common power, mounting, thermal and software causes can affect multiple channels simultaneously. Sensor fusion estimates a common state from measurements with different error characteristics. Fault detection should occur before suspect measurements are allowed to dominate the estimate.

4.4 Watchdog and Safe-State Design

A watchdog timer detects software that fails to refresh a monitored heartbeat. It is useful for detecting a stalled task or processor lockup, but it does not prove that the computed control output is correct. Therefore, watchdog supervision should be combined with sensor health checks, task timing checks and a documented recovery policy.

4.5 Research Gap

A common educational implementation demonstrates stable flight using a controller and sensors, but may not provide a clearly documented chain from fault detection to isolation, severity assessment,

safe-state selection and evidence-based validation. This project addresses that gap through a modular framework and a repeatable fault-injection test plan.

4.6 Summary

The literature and platform documentation support a layered approach: use a proven baseline flight controller, add transparent health monitoring, define conservative supervisory actions, and validate each fault scenario independently. The project should not claim certification or guaranteed flight safety from a classroom prototype.

4.7 Comparison of Diagnostic Approaches

Threshold-based diagnosis is easy to implement and explain, making it suitable for an initial student prototype. Its limitation is that a fixed threshold may be sensitive to operating conditions and sensor noise. Persistence counters and hysteresis can reduce unnecessary alarms, but limits still require calibration and validation.

Model-based diagnosis compares measured signals with values predicted by a mathematical model. It can detect subtle inconsistencies, but performance depends on model accuracy and parameter identification. Statistical approaches examine signal distributions or trends and may identify gradual changes, although they require representative data and carefully selected decision limits.

For this project, transparent range, timeout and consistency checks are selected as the baseline. More advanced methods are considered future extensions after the basic framework has been verified.

CHAPTER 5: HARDWARE AND SOFTWARE IMPLEMENTATION

5.1 Proposed Hardware Configuration

The final component selection must be confirmed by the project team and supervisor. The following is a reference bill of materials for planning; it is not a statement that these components have already been purchased or tested.

Component	Suggested specification / role	Quantity
Flight controller	Educational autopilot board with documented sensor and failsafe interfaces	1
Primary IMU	3-axis gyroscope + 3-axis accelerometer, integrated or external	1
Secondary IMU	Independent IMU channel if supported by platform	1
Barometer	Digital pressure sensor supported by controller	1
GNSS receiver	Position and velocity source for outdoor tests	1
Power monitor	Battery voltage/current telemetry	1
Battery	Manufacturer-approved battery matched to platform	1
Telemetry / USB interface	For status monitoring and bench configuration	1
Test computer	Simulation, logging and analysis	1

5.2 Software Tools

Tool	Purpose
PX4 or ArduPilot (select one)	Baseline flight-control firmware and health/failsafe status
SITL simulator	Software-in-the-loop testing without a flying vehicle
Python	Log parsing, test automation and result plotting
QGroundControl / Mission Planner	Configuration and health-status observation, as supported
Version control	Track source changes and test configurations

5.3 Proposed Sensor and Monitor Parameters

Parameter	Reference value for initial simulation	Validation note
Gyroscope range	±2000 °/s	Use actual sensor configuration
Accelerometer range	±16 g	Use actual sensor configuration

IMU sample frequency	200 Hz	Confirm supported driver rate
Barometer update rate	25 Hz	Depends on selected device
GNSS update rate	5 Hz	Depends on receiver configuration
Battery warning threshold	3.5 V/cell (illustrative)	Must follow battery manufacturer limits and loaded-voltage behaviour
Battery critical threshold	3.3 V/cell (illustrative)	Do not use as a universal cutoff; configure to battery specification
IMU stale timeout	100 ms	Simulation acceptance target
Heartbeat timeout	100 ms	Simulation acceptance target

5.4 Fault Detection Logic

The monitoring module performs the following checks:

Range check: confirm that each measurement is within the configured sensor limits.

Freshness check: calculate sample age from the timestamp and reject stale data.

Finite-value check: reject NaN, infinity and malformed data.

Freeze check: detect a value that remains unchanged beyond a permitted duration when change is expected.

Consistency check: compare independent channels after calibration and time alignment.

Heartbeat check: verify that the monitored software task reports progress within the allowed interval.

Power check: classify low-voltage warnings using the selected battery and power-system specifications.

5.5 Fault Event Record

Field	Example format
Timestamp	2026-11-04 14:32:10.250
Source	IMU_2
Fault type	STALE_DATA
Severity	DEGRADED
Evidence	Sample age = 125 ms; limit = 100 ms
Action	Mark channel invalid; notify supervisor
Recovery status	Pending / recovered / latched

5.6 Software Module Description

Module	Main responsibility
sensor_interface	Read measurements and timestamps from platform interface
validate_sample	Range, freshness, finite-value and freeze checks
compare_channels	Calculate disagreement between comparable channels
fault_manager	Track persistence counters and classify faults
safety_supervisor	Manage NORMAL, DEGRADED, RECOVERY and SAFE states
event_logger	Store fault events and periodic health summary

5.7 Implementation Note

The actual code must use the supported APIs of the chosen flight stack. Do not connect an independent microcontroller directly to motor outputs or override flight-controller safety

mechanisms without supervisor approval. The safest academic implementation is to observe health data and test supervisor decisions in simulation first.

5.8 Integration and Verification Plan

Integration should proceed in stages. First, verify that each sensor interface produces correctly scaled and timestamped data. Next, test each validation function using normal, boundary and invalid input samples. Then verify fault classification and event logging using a software test harness. Only after these stages should the monitor be connected to the supported status interface of the selected flight controller.

The project team should maintain a configuration record containing hardware model numbers, firmware version, sensor configuration, software revision and test date. This record is essential because a change in firmware or sensor update rate may affect timeout settings and expected diagnostic behaviour.

The implementation should not bypass the flight controller's built-in arming checks or failsafe mechanisms. The proposed monitor is an additional observation and supervisory layer, and its interface must remain within the documented capabilities of the selected platform.

CHAPTER 6: FLOWCHART AND ALGORITHM

6.1 Overall Algorithm

Initialise the monitoring modules and load validated sensor limits.

Read the latest sensor values, timestamps, controller status and power status.

Check whether each sample is finite, within range and sufficiently recent.

Compare independent channels only when both are valid and time-aligned.

Update persistence counters for each detected abnormal condition.

If the condition clears, reduce the counter using hysteresis; otherwise increase it.

Classify the fault as advisory, degraded or critical according to the fault table.

Notify the safety supervisor and record the evidence in the event log.

Select the predefined response permitted for the current flight state.

Continue monitoring and record whether the system recovers or remains in a latched safe state.

6.2 Pseudocode

```
INITIALISE monitor, counters, event_log
state = NORMAL

LOOP every 20 ms:
    samples = acquire_health_data()

    FOR each critical sample:
        valid = finite(sample) AND within_range(sample)
        fresh = sample_age(sample) <= configured_timeout
        update_channel_health(valid AND fresh)

    IF two comparable channels are valid and time-aligned:
        error = absolute_difference(channel_1, channel_2)
        update_disagreement_counter(error > tolerance)

    check_controller_heartbeat()
    check_power_status()
    classify_confirmed_faults()

    IF critical fault is confirmed:
```

```
                        state = SAFE_OR_RECOVERY according to approved policy
                          ELSE IF degraded fault is confirmed:
                                                 state = DEGRADED
                                                    ELSE:
                                                   state = NORMAL

                                              log_state_change_and_fault_evidence()
                                                      publish_health_status(state)
END LOOP
```

6.3 Example Threshold Logic

For the simulation model, the following illustrative logic may be used. A fault is not declared from one abnormal reading; it must persist for the configured number of samples unless the condition is an immediate platform-reported critical fault.

Monitor	Illustrative condition	Decision
IMU freshness	Age > 100 ms	Stale-data flag
Gyro consistency	Difference > 5 °/s for 5 samples	Disagreement flag
Heartbeat	No update for > 100 ms	Task-health fault
Battery warning	Below configured warning threshold	Warning state
Invalid measurement	NaN / infinity / outside configured range	Reject sample and log

6.4 Flow Sequence (Text Form)

```
START → Initialise sensors and monitor → Acquire health data → Validate samples → Compare channels → Check heartbeat and power → Classify fault → Select supervisor state → Log and publish status → Repeat.
```

6.5 Safety Design Principle

A fault-tolerant framework must fail conservatively. If sensor validity or control integrity is uncertain, the software must not invent a replacement measurement or continue as if the system were healthy. The selected response must be defined, reviewed and tested for the actual platform.

6.6 Recovery and Event-Latching Logic

A detected fault should not automatically be cleared merely because one later sample appears normal. Recovery logic should require a configured number of consecutive valid samples and confirmation that relevant health indicators have returned to their permitted ranges. This prevents rapid switching between normal and degraded states when a signal is intermittent.

Some events may be latched for review even after the signal recovers. A latched record preserves the original evidence and allows the operator or reviewer to acknowledge the event according to project policy. Current health and historical fault records should remain distinct.

CHAPTER 7: SIMULATION, TESTING AND RESULT ANALYSIS

7.1 Test Setup

The recommended first stage is software-in-the-loop (SITL), followed by bench testing with propellers removed. Faults should be injected through simulation parameters or recorded-data replay. This avoids exposing people or equipment to an unvalidated aircraft. Flight testing, if later approved, must be conducted by qualified supervisors in a suitable controlled area.

7.2 Test Scenarios

Test ID	Injected condition	Expected framework response
T1	Normal sensor streams	Remain in NORMAL; no fault event
T2	IMU sample stream stops	Flag stale data within 100 ms target
T3	One gyro channel has constant bias	Disagreement detected after persistence threshold
T4	GNSS packets delayed or missing	Mark GNSS unavailable; notify supervisor
T5	Monitoring-task heartbeat stops	Watchdog/task-health fault detected
T6	Battery status crosses configured warning level	Issue power warning and log event
T7	Transient one-sample spike	Do not change mode if persistence condition is not met
T8	Fault clears after detection	Record recovery; return only according to approved reset policy

7.3 Performance Metrics

Metric	Definition	Target for simulation
Detection latency	Time from injected fault to fault flag	≤ 100 ms for IMU timeout; ≤ 200 ms supervisor transition
False-alarm rate	False detections / total healthy test windows	Below 1% in the selected test dataset
Data availability	Valid samples / expected samples × 100	At least 99% in nominal simulation
Safe-state transition success	Successful expected transitions / confirmed critical tests × 100	100% in defined test cases
Recovery classification	Correctly logged recovery events / injected recoveries × 100	100% in defined test cases

7.4 Illustrative Calculation (Not Measured Results)

Assume a simulated IMU data stream runs at 200 Hz. The nominal sample interval is:

$T = 1 / 200 = 0.005 \text{ s} = 5 \text{ ms}$

If the stale-data timeout is configured as 100 ms, the timeout corresponds to:

Number of missed sample intervals = 100 ms / 5 ms = 20 intervals

Assume a fault is injected at t = 2.000 s and the framework flags it at t = 2.085 s. The illustrative detection latency is:

Detection latency = 2.085 − 2.000 = 0.085 s = 85 ms

For a test set containing 100 fault injections, if 96 are detected correctly, the detection rate is:

Detection rate = (96 / 100) × 100 = 96%

These figures demonstrate the calculation method only. Replace them with actual simulator logs and test observations before submitting the final report.

7.5 Result Table Template

Test ID	Measured latency	Detected?	False alarm?	Observed state	Remarks
T1	_____	_____	_____	_____	_____
T2	_____	_____	_____	_____	_____
T3	_____	_____	_____	_____	_____
T4	_____	_____	_____	_____	_____
T5	_____	_____	_____	_____	_____
T6	_____	_____	_____	_____	_____
T7	_____	_____	_____	_____	_____
T8	_____	_____	_____	_____	_____

7.6 Analysis

The framework should be considered successful only when each injected fault is detected within its defined time limit, the event is classified correctly, and the supervisor selects the expected state without unnecessary mode switching during healthy operation. The persistence counter helps reduce false alarms caused by transient noise. The test results should also identify faults that cannot be isolated because the available measurements are insufficient.

7.7 Limitations of Evaluation

SITL and bench testing cannot fully reproduce vibration, electromagnetic interference, propeller loading, thermal effects, radio-frequency interference or all common-cause failures. Therefore, simulation success is evidence of software behaviour under the selected model, not proof of operational flight safety.

7.8 Test Procedure and Documentation

Before each test, record the software version, simulator configuration, initial system state and expected response. Run a healthy baseline test first so normal timing and signal variation are documented. Introduce one fault condition at a time, repeat the test under the same configuration and save the generated log file.

For each run, calculate detection latency from the recorded fault-injection time and first valid detection time. Record whether the correct fault class was assigned, whether the supervisor entered the expected state and whether an alarm occurred during a healthy interval. Repeat selected tests to identify inconsistent behaviour.

A result should be reported as measured only when it comes from an actual simulator or bench log. Values used to explain formulas or demonstrate the report format must remain clearly labelled as illustrative. A test that cannot be completed should be marked as not tested rather than assigned an assumed result.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project report presents a modular framework for fault-tolerant drone flight control. The design combines sensor validation, timeout monitoring, channel-consistency checks, watchdog supervision, fault classification, event logging and supervisory safe-state management. Rather than replacing the baseline flight controller, the framework observes system health and supports predefined degraded or recovery responses.

The report also defines a repeatable evaluation plan using software-in-the-loop and propeller-removed bench tests. Proposed parameter values and example calculations provide a starting point for implementation; they must be replaced or confirmed using the selected hardware datasheets and measured test logs. The central contribution is a structured method for linking fault evidence to a documented supervisory response.

8.2 Key Expected Outcomes

A modular architecture separating normal flight control from health monitoring and supervisory decisions.

A documented fault catalogue covering sensor, communication, processor and power faults.

Detection of stale, invalid and inconsistent sensor data using transparent checks.

Event logs containing fault source, timestamp, evidence, severity and response.

A repeatable SITL and bench-test procedure with measurable acceptance criteria.

8.3 Future Scope

Add model-based residual generation using a validated vehicle and sensor model.

Use statistical change detection to identify slow sensor bias and noise growth.

Introduce dissimilar redundant sensing and independent power monitoring to reduce common-cause failures.

Integrate health status with a ground-station dashboard and long-term log analysis.

Evaluate fault-tolerant estimation methods such as robust Kalman filtering under controlled simulation.

Perform formal hazard analysis and requirements traceability for a larger research prototype.

Validate the design with hardware-in-the-loop testing before any supervised flight evaluation.

8.4 Project Deliverables

The planned deliverables are a documented system architecture, a fault catalogue, data-validation and diagnostic rules, a supervisory-state definition, an event-log format and a repeatable test procedure. The team should also retain source code, configuration files, simulator logs and photographs of the completed bench setup as supporting project evidence.

The final evaluation should compare each requirement with its corresponding test case. Any requirement that has not been verified should be identified as pending. This traceability distinguishes functions proposed in the design from functions demonstrated by the completed prototype.

CHAPTER 9: REFERENCES

[1] PX4 Autopilot. (2025). PX4 User Guide: Flight Controller, Sensor Calibration, and Failsafe Configuration. https://docs.px4.io/

[2] ArduPilot Development Team. (2025). ArduPilot Copter Documentation: Failsafe and Pre-Arm Checks. https://ardupilot.org/copter/

[3] Beard, R. W., & McLain, T. W. (2012). Small Unmanned Aircraft: Theory and Practice. Princeton University Press.

[4] Stevens, B. L., Lewis, F. L., & Johnson, E. N. (2015). Aircraft Control and Simulation (3rd ed.). Wiley.

[5] Isermann, R. (2006). Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance. Springer.

[6] Blanke, M., Kinnaert, M., Lunze, J., & Staroswiecki, M. (2016). Diagnosis and Fault-Tolerant Control (3rd ed.). Springer.

[7] IEEE. (2010). IEEE Std 1012-2010: IEEE Standard for System and Software Verification and Validation. IEEE.

[8] RTCA. (2011). DO-178C: Software Considerations in Airborne Systems and Equipment Certification. RTCA. (Referenced for background only; this student project is not certified to DO-178C.)

[9] Microchip Technology. (2024). Watchdog Timer Design and Embedded System Reliability Application Notes. https://www.microchip.com/

[10] Texas Instruments. (2024). Inertial Measurement Unit and Sensor Signal-Conditioning Technical Resources. https://www.ti.com/

APPENDIX A: PROPOSED PARAMETER SHEET

The following values are preliminary simulation parameters. Final values must be selected from the exact component datasheets and the chosen flight stack's supported configuration.

Parameter	Initial value	Units / note
IMU sample rate	200	Hz; simulation assumption
Health-monitor rate	50	Hz
IMU timeout	100	ms
Heartbeat timeout	100	ms
Gyro disagreement threshold	5	°/s; illustrative
Persistence count	5	consecutive samples
Supervisor transition target	200	ms after confirmed critical fault
GNSS update rate	5	Hz; receiver-dependent
Barometer update rate	25	Hz; sensor-dependent
Battery warning level	3.5	V/cell; illustrative only
Battery critical level	3.3	V/cell; illustrative only

APPENDIX B: TEST RECORD TEMPLATE

Item	Record
Test date / time	_____
Platform / firmware version	_____
Simulation or bench setup	_____
Fault injected	_____
Injection time	_____
Detection time	_____
Detection latency	_____
Supervisor state selected	_____
Expected response achieved?	YES / NO
Log file name	_____
Test engineer / reviewer	_____
Comments	_____

APPENDIX C: ITEMS TO COMPLETE BEFORE FINAL SUBMISSION

Confirm the official project course code, semester, academic year and guide names with the department.

Insert the actual flight-controller model, sensor part numbers and photographs of the prototype.

Replace all illustrative values with values supported by datasheets or your own design decisions.

Run SITL/bench tests and fill in the result table with real observations.

Add your own architecture diagram, flowchart, wiring diagram and screenshots from the actual project.

Update the table of contents and page numbers in Microsoft Word.